



第4章 线程

- 本章内容
 - 1. 线程的概念
 - 2. 多线程模型
 - 3. 线程库





动机

■ 多进程协作的问题

■ 基本的问题：内存空间独立

- 回顾fork(), 进程间天然互相不可见（后续学习原理）

■ 进程的创建、撤消、切换等操作涉及到资源（特别是内存资源）管理，时空开销较大

- 随着后续学习逐步理解

■ 多个进程间的协作不方便

- 想要共享数据，必须通过进程间通信，创建共享内存，或借助消息、管道等机制





动机

■ 进程具有两个属性

- 拥有资源的独立单位
- 调度和分派的基本单位

■ 引入线程的概念后

- 调度及分派单位称为线程或轻型进程
- 拥有资源的单位称为进程或任务
- 进程变成户口，线程代表个人
 - CPU分配给个人执行
 - 各种资源分配给进程，供其中线程共享

启动一个应用程序，即代表启动了一个包含一个线程的进程

1) 该线程从main()函数开始执行

2) 随着程序执行，更多线程可被创建





传统的进程怎么办

- 主要针对Unix/Linux
 - 早期没有线程，大量传统程序使用fork()、wait()书写，有完备的多进程同步、通信机制
 - 引入线程，内核变化，须确保早期程序被兼容
- 重型进程：等价于只有一个线程的任务
- 新问题
 - 如果程序中既使用多线程，又用fork()创建进程，则涉及一些策略和细节问题，系统须谨慎处理
 - 本课程忽略，有兴趣者自学





线程定义

- 线程是CPU使用的一个基本单元，包括
 - 线程标识
 - 程序计数器
 - 寄存器集
 - 栈
- 线程与属于同一进程的线程共享
 - 代码段
 - 数据段
 - 其他操作系统资源

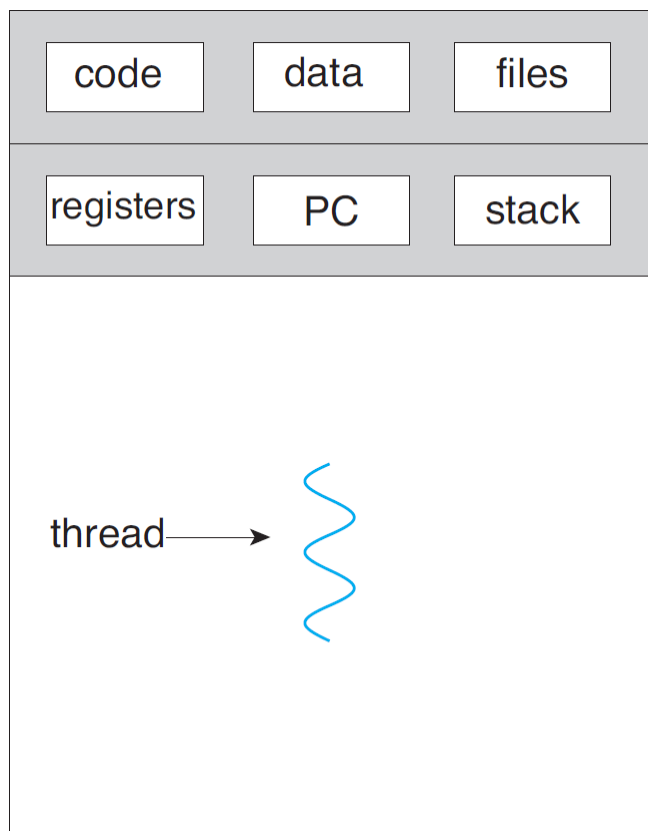
同一进程下的线程共享全局变量、堆、打开的文件，甚至借用全局指针也可互访局部变量

多进程共享资源有避免错乱的问题（即同步问题），多线程同样存在，后续章节讨论

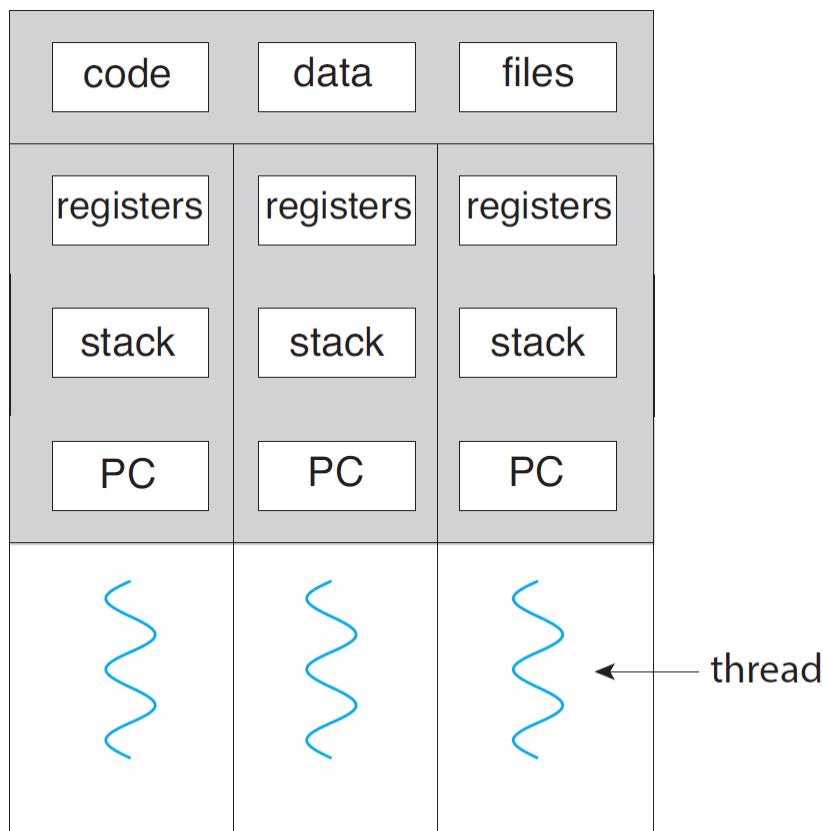




单线程进程和多线程进程



single-threaded process



multithreaded process



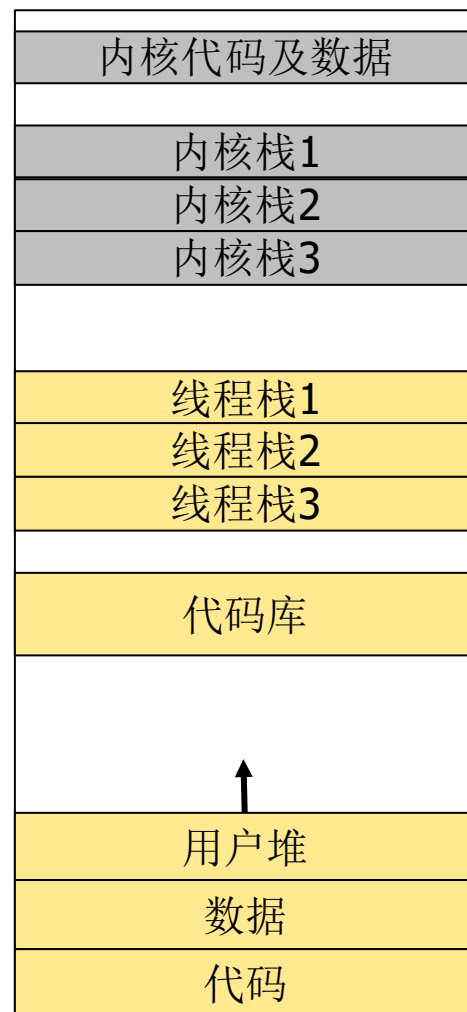


■ 独立的内核栈和用户栈

- 每个线程都有自己的栈，用于存放临时数据
- 相应的，每个线程有内核栈

■ 共享其它区域

- 除栈以外的其它区域，进程内的所有栈共享
- 多个线程需要动态分配内存时，将在同一个堆上完成





线程的状态

■ 线程状态

- 创建、就绪、运行、阻塞、终止
- 进程只剩创建、撤销、活动、挂起等状态
 - 进程的挂起及终止将影响到其中的所有线程

■ 进程有PCB，线程有TCB

- 线程标识符、状态、CPU寄存器值、优先级、指向PCB的指针

■ 线程的同步与通信与进程类似

- 但免除了很多通信的必要





优点

■ 响应性

- 进程在某个部分被阻塞的情况下被允许继续执行，对于用户界面尤其重要

■ 资源共享

- 天然共享资源，比进程通信方便，免除系统调用

■ 经济性

- 不涉及内存资源管理，创建开销小，切换开销低

■ 可扩展性

- 使单个进程可以利用多核架构的优势





第4章 线程

- 本章内容
 - 1. 线程的概念
 - 2. 多线程模型
 - 3. 线程库



多线程模型

- 操作系统中有多种方式实现对线程的支持
 - 内核线程
 - 即真正的线程，由OS内核创建并管理
 - 所有当代OS均支持内核线程
 - 用户线程
 - 不依赖于OS核心，由应用进程利用线程库提供创建、同步、调度和管理（不依赖系统调用）
 - 维护由应用进程完成，可以用于不支持内核线程的操作系统
 - 上述两种方法的组合实现





两种方法的组合

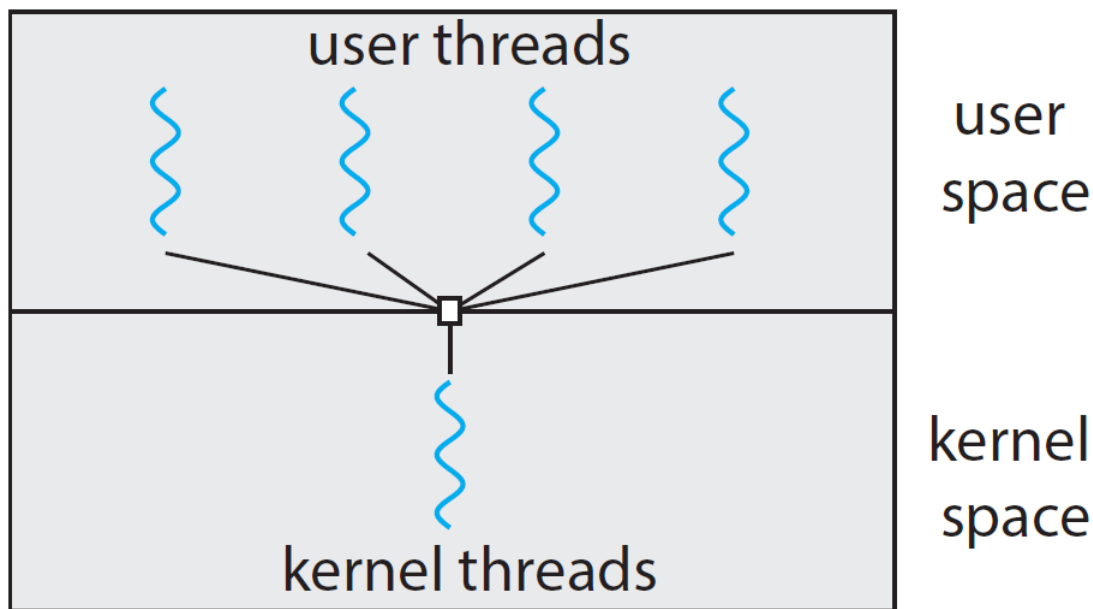
- 内核支持多线程的建立、调度与管理
- 同时，系统中又提供线程库，允许应用程序建立、调度和管理用户线程
- 组合关系
 - 多对一
 - 一对一
 - 多对多





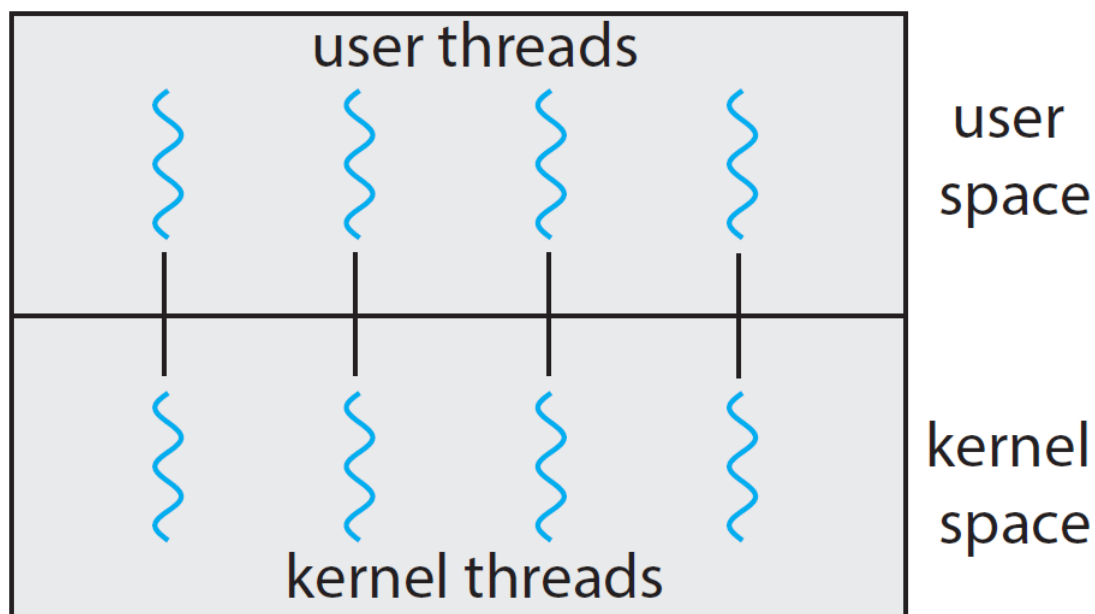
多对一模型

- 将多个用户线程映射到一个内核线程
 - 线程管理由线程库在用户空间进行
 - 线程是手搓的，系统调用和阻塞却是真实的，当一个线程阻塞时，整个进程都必须等待



一对一模型

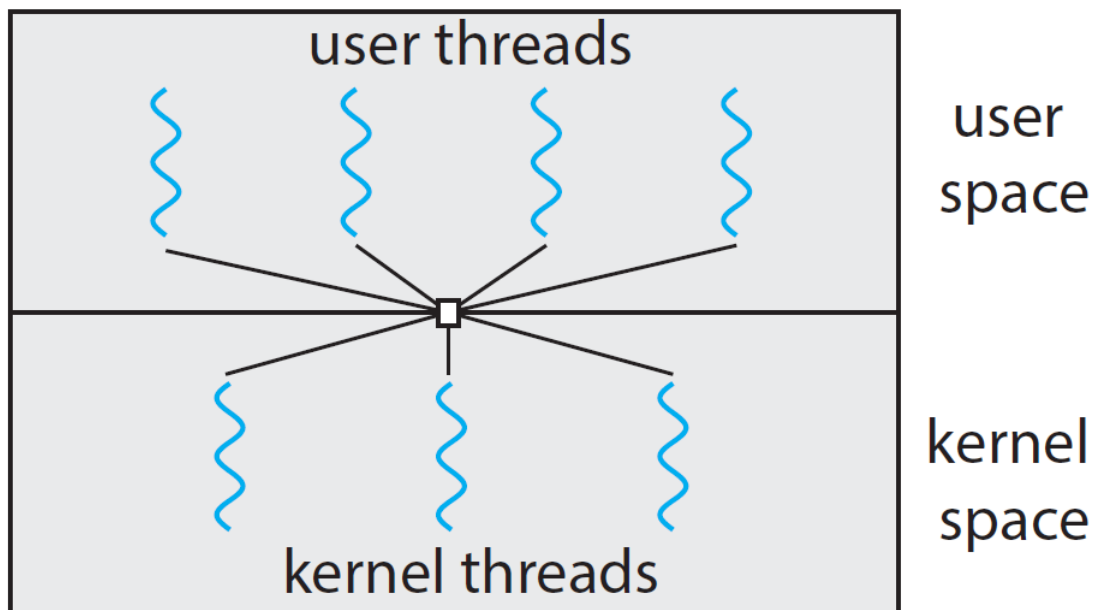
- 将每个用户线程映射到一个内核线程
 - 比多对一模型提供更好的并发性
 - 但一个进程中的线程数量通常受系统限制
 - 主流OS均属一对一模型





多对多模型

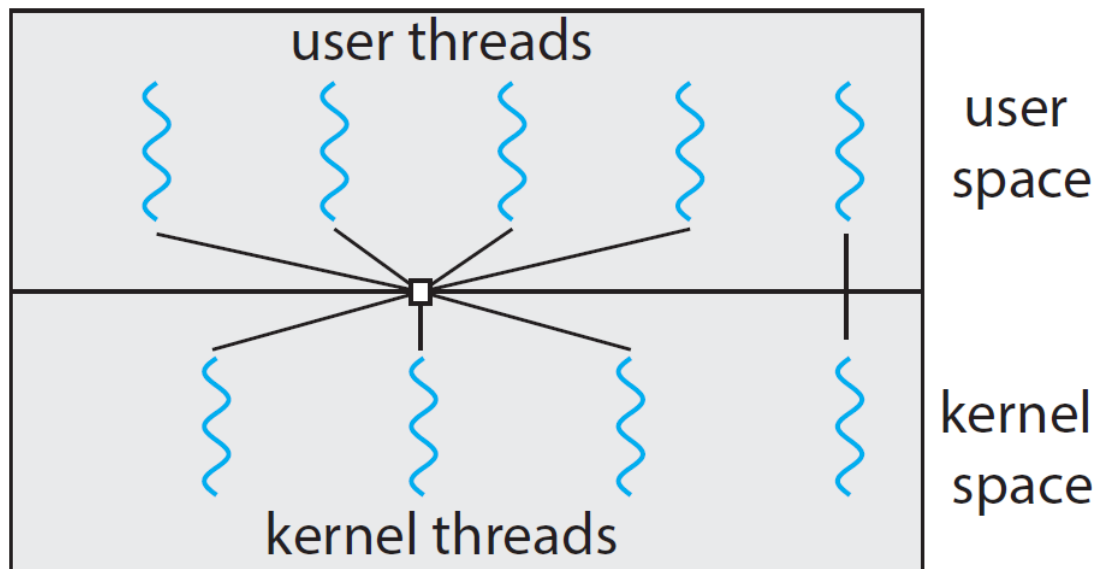
- 多路复用M个用户线程到N个内核线程上
 - $M \geq N$
 - 如果某用户线程阻塞等待，则创建另一个内核线程，同类的用户线程被转移到新的内核线程





多对多模型

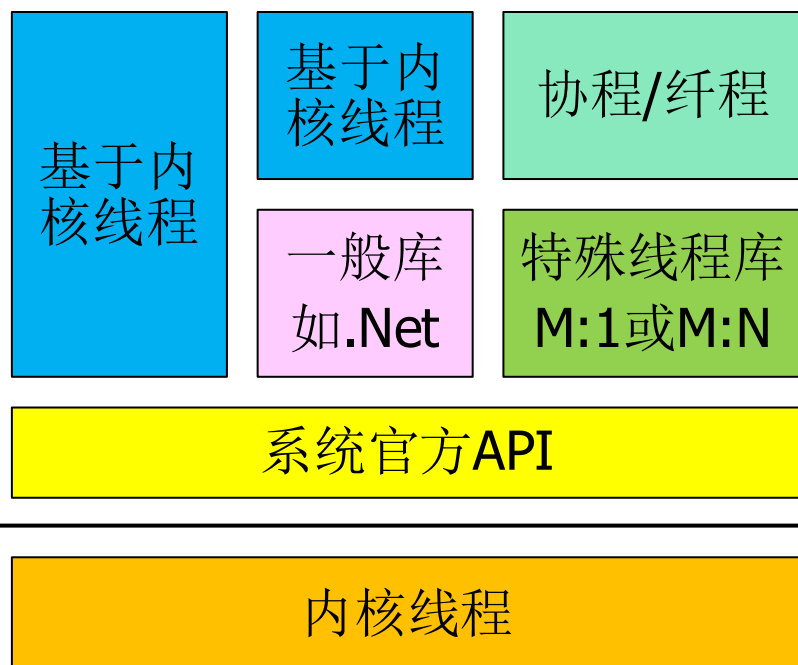
- 多路复用M个用户线程到N个内核线程上
 - 更灵活的变种：用户线程既可以动态分享内核线程，也可以独占一个内核线程



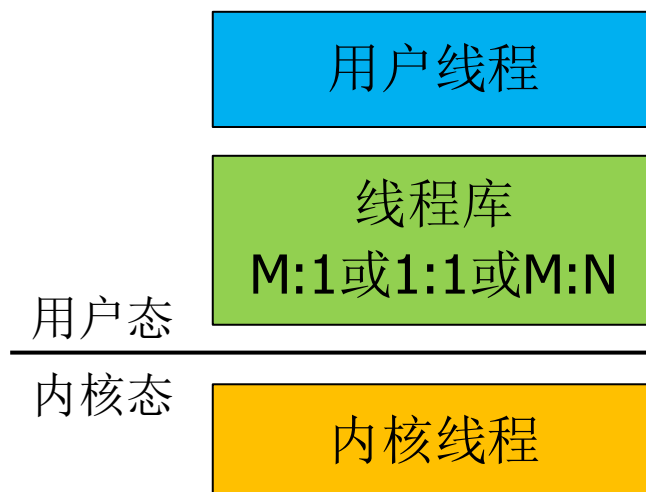


现实情况

- 线程就是内核线程，多数情况下满足需要
- 手搓的用户线程通常称为**协程/纤程**，依靠特殊线程库，用于特殊场合



现实情况



教材描述的情况





第4章 线程

- 本章内容
 - 1. 线程的概念
 - 2. 多线程模型
 - 3. 线程库





POSIX pthreads

- 只是一种规范，而非实现
 - pthreads只对接口进行了定义，不同的OS根据自己的需求提供了实现
 - 各种类Unix系统（包括Linux和Mac OS X）均实现了pthreads规范
 - pthread_create(), pthread_exit()
 - pthread_join(), pthread_detach()





POSIX pthreads

■ Linux案例

共享

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
int sum;
void *runner(void *param) {
    int i, upper = atoi(param);
    sum = 0;
    for (i = 1; i <= upper; i++) sum += i;
    pthread_exit(0);
}
int main(int argc, char *argv[]) {
    pthread_t tid;
    pthread_attr_t attr;
    pthread_attr_init(&attr);
    pthread_create(&tid, &attr, runner, argv[1]);
    pthread_join(tid, NULL);
    printf("sum = %d\n", sum);
}
```





POSIX pthreads

■ Linux案例

- 多个子线程，主线程等待其逐个结束

```
#define NUM_THREADS 10
.....
pthread_t workers[NUM_THREADS];
.....
for (int i = 0; i < NUM_THREADS; i++)
    pthread_join(workers[i], NULL);
```





POSIX pthreads

- 进程结束的条件
 - `main()`执行到`return`
 - 所有非分离线程结束
 - 任何线程调用`exit()`
- 自学
 - `pthread_self()`
 - `pthread_cancel()`
 - `pthread_attr_init()`, `pthread_attr_destroy()`





POSIX pthreads

■ MacOS的线程库

- 兼容POSIX，支持pthreads库
- 早期主流NSThread，对pthreads的封装
- 目前主流GCD，旨在替换NSThread，简化多线程编程，隐藏线程的概念及其细节
- NSOperation，对GCD面向对象封装和加强

■ 特殊

- Linux最初（2.4及之前）只有进程，使用特殊手段支持pthreads，但有些问题，说不清到底算不算内核线程，2.6才支持内核线程





Win32

```
#include <windows.h>
#include <stdio.h>
DWORD Sum;
DWORD WINAPI Summation(LPVOID Param) {
    DWORD Upper = *(DWORD*)Param;
    for (DWORD i = 1; i <= Upper; i++) Sum += i;
    return 0;
}
int main(int argc, char *argv[]) {
    DWORD ThreadId;
    HANDLE ThreadHandle;
    int Param = atoi(argv[1]);
    ThreadHandle = CreateThread(NULL, 0,
        Summation, &Param, 0, &ThreadId);
    WaitForSingleObject(ThreadHandle, INFINITE);
    CloseHandle(ThreadHandle);
    printf("sum = %d\n", Sum);
}
```

共享





C++标准库

■ `std::thread`类

- 构造函数, `get_id()`, `join()`, `detach()`
- 自学: `joinable()`, `native_handle()`

■ `std::this_thread`命名空间

- `get_id()`, `sleep_for()`, `sleep_until()`
- 自学: `yield()`





C++标准库

```
#include <stdio.h>
#include <stdlib.h>
#include <chrono>
#include <iostream>
#include <thread>
void thread_task(int n) {
    std::this_thread::sleep_for(std::chrono::seconds(n));
    std::cout << "thread " << std::this_thread::get_id()
              << " paused " << n << " seconds" << std::endl;
}
int main(int argc, const char *argv[]) {
    std::thread threads[5];
    std::cout << "Creating 5 threads...\n";
    for (int i = 0; i < 5; i++)
        threads[i] = std::thread(thread_task, i + 1);
    std::cout << "Now wait for them to join\n";
    for (auto& t: threads) t.join();
    std::cout << "All threads joined.\n";
    return EXIT_SUCCESS;
}
```





其它（自学）

■ 其它语言的标准库

■ Java

- Thread类，Runnable接口，ThreadPoolExecutor类
- Callable接口和FutureTask类

■ C# (.Net)

- Thread类，ThreadPool类，Task类，Parallel类

■ Python

- threading.Thread类，thread模块
- concurrent.futures.ThreadPoolExecutor类
- Python中一个进程的多个线程被限定在一个CPU上





其它（自学）

- 协程/纤程库：用于大量并发轻量任务
 - Go
 - Goroutines模块，协程，多对多
 - Win32
 - Fibers库，多对一
 - UMS（User-Mode Scheduling）库，多对多
 - Python
 - greenlet模块，协程，多对一
 - Java
 - Quasar库，Fiber类，多对多





练习

- 4.6 What resources are used when a thread is created?
How do they differ from those used when a process is created?
- 4.10 Which of the following components of program state are shared across threads in a multithreaded process?
 - a. Register values
 - b. Heap memory
 - c. Global variables
 - d. Stack memory





考研题1

- 在支持多线程的系统中，进程P创建的若干个线程不能共享的是_____。 11
- A、进程P的代码段
- B、进程P中打开的文件
- C、进程P的全局变量
- D、进程P中某线程的栈指针





考研题2

- 下列关于进程和线程的叙述中，正确的是（ ）
12
 - A. 不管系统是否支持线程，进程都是资源分配的基本单位
 - B. 线程是资源分配的基本单位，进程是调度的基本单位
 - C. 系统级线程和用户级线程切换都需要内核的支持
 - D. 同一进程的各个线程拥有各自不同的地址空间

